

Funkcje MS SQL



CASE

Ocenia listę warunków i zwraca jedno z wielu możliwych wyrażeń wynikowych.

Wyrażenie CASE ma dwa formaty:

- Proste wyrażenie CASE porównuje wyrażenie z zestawem prostych wyrażeń w celu określenia wyniku.
- Przeszukiwane wyrażenie CASE ocenia zestaw wyrażeń logicznych w celu określenia wyniku.

Oba formaty obsługują opcjonalny argument ELSE.

CASE może być używany w dowolnej instrukcji lub klauzuli, która pozwala na prawidłowe wyrażenie.

- Na przykład można użyć CASE w instrukcjach takich jak SELECT, UPDATE, DELETE i SET oraz w klauzulach takich jak <select_list>, IN, WHERE, ORDER BY i HAVING.
- SQL Server pozwala tylko na 10 poziomów zagnieżdżenia w wyrażeniach CASE
- Wyrażenie CASE ocenia swoje warunki sekwencyjnie i zatrzymuje się na pierwszym warunku, którego warunek jest spełniony

CASE

```
SELECT
    last_name,
    salary,
    CASE
        WHEN salary > 10000 THEN
            'OK'
        ELSE
            '---'
    END AS warunek
FROM
    employees
```

	last_name	salary	warunek
1	King	24000.00	OK
2	Kochhar	17000.00	OK
3	De Haan	17000.00	OK
4	Hunold	9000.00	---
5	Ernst	6000.00	---
6	Austin	4800.00	---
7	Pataballa	4800.00	---
8	Lorentz	4200.00	---

```
SELECT ProductNumber,
    Category = CASE ProductLine
        WHEN 'R' THEN 'Road'
        WHEN 'M' THEN 'Mountain'
        WHEN 'T' THEN 'Touring'
        WHEN 'S' THEN 'Other sale items'
        ELSE 'Not for sale'
    END,
    Name
FROM Production.Product
```

	ProductNumber	Category	Name
406	FR-T67Y-54	Touring	LL Touring Frame - ...
407	FR-T67Y-58	Touring	LL Touring Frame - ...
408	FR-T67U-44	Touring	LL Touring Frame - ...
409	FR-M63S-40	Mountain	ML Mountain Frame...
410	FR-M63S-42	Mountain	ML Mountain Frame...
411	FR-M63S-46	Mountain	ML Mountain Frame...
412	RB-9231	Not for sale	Rear Brakes
413	SE-M236	Mountain	LL Mountain Seat/S...
414	SE-M798	Mountain	ML Mountain Seat/...
415	SE-M940	Mountain	HL Mountain Seat/...
416	SE-R581	Road	LL Road Seat/Saddle



CASE

```
select
    department_id,
    department_name
from departments
order by
case department_id
    when 30 then 19
    when 40 then 18
    else department_id end
```

	department_id	department_name
1	10	Administration
2	40	Human Resources
3	30	Purchasing
4	20	Marketing
5	50	Shipping

IIF

IIF to skrócony sposób zapisu wyrażenia CASE, zwraca jedną z dwóch wartości, w zależności od tego, czy wyrażenie logiczne ma wartość true czy false w SQL Server.

Select

last_name,
salary,

IIF(salary>15000, 'OK', '-') warunek

from employees

	last_name	salary	warunek
1	King	24000.00	OK
2	Kochhar	17000.00	OK
3	De Haan	17000.00	OK
4	Hunold	9000.00	-
5	Ernst	6000.00	-
6	Austin	4800.00	-
7	Pataballa	4800.00	-

SELECT [Result] = IIF(45 > 30, '45>30', null);

	Result
1	45>30

NULLIF

Zwraca wartość null, jeśli dwa określone wyrażenia są równe.

Na przykład `SELECT NULLIF(4,4) AS Same, NULLIF(5,7) AS Different;` zwraca NULL dla pierwszej kolumny (4 i 4), ponieważ dwie wartości wejściowe są takie same.

Druga kolumna zwraca pierwszą wartość (5), ponieważ dwie wartości wejściowe są różne.

```
select  
    nullif(4,4),  
    nullif(5,7)
```

	(No column name)	(No column name)
1	NULL	5

CHOOSE

Zwraca element w określonym indeksie z listy wartości w SQL Server.

```
SELECT CHOOSE ( 3, 'Manager', 'Director', 'Developer', 'Tester' ) AS Result;
```

	Result
1	Developer

```
SELECT CHOOSE ( 1, 'Manager', 'Director', 'Developer', 'Tester' ) AS Result;
```

	Result
1	Manager

GREATEST LEAST

GREATEST - Zwraca element w określonym indeksie z listy wartości w SQL Server.

```
SELECT GREATEST(1, 10, 35) AS GREATESTVal;
```

	GreatestVal
1	35

LEAST - Ta funkcja zwraca minimalną wartość z listy jednego lub więcej wyrażeń.

```
SELECT LEAST(1, 10, 35) AS LEASTVal;
```

	LeastVal
1	1

CONCAT

Funkcja ta zwraca ciąg znaków będący wynikiem konkatencji lub połączenia dwóch lub więcej wartości ciągu znaków.

```
select
    first_name,
    last_name,
    CONCAT(first_name, last_name) fn,
    CONCAT(first_name, ' ', salary, ', dept: ', department_id) fsd
from employees
```

	first_name	last_name	fn	fsd
1	Steven	King	StevenKing	Steven 24000.00, dept: 90
2	Neena	Kochhar	NeenaKochhar	Neena 17000.00, dept: 90
3	Lex	De Haan	LexDe Haan	Lex 17000.00, dept: 90
4	Alexander	Hunold	AlexanderHunold	Alexander 9000.00, dept: 60
5	Bruce	Ernst	BruceErnst	Bruce 6000.00, dept: 60
6	David	Austin	DavidAustin	David 4800.00, dept: 60
7	Valli	Pataballa	ValliPataballa	Valli 4800.00, dept: 60

CONCAT_WS

Ta funkcja zwraca ciąg znaków będący wynikiem konkatenacji lub połączenia dwóch lub więcej wartości ciągów. Oddziela ona te połączone wartości łańcuchowe ogranicznikiem określonym w pierwszym argumencie funkcji

```
select  
    CONCAT_WS('---',first_name, last_name) fn  
from employees
```

	fn
1	Steven--King
2	Neena--Kochhar
3	Lex--De Haan
4	Alexander--Hunold
5	Bruce--Ernst
6	David--Austin
7	Valli--Pataballa

LOWER, UPPER

Zwraca wyrażenie znakowe po przekonwertowaniu wielkich liter na małe(LOWER) lub odwrotnie(UPPER).

	last_name	lower	upper
1	King	king	KING
2	Kochhar	kochhar	KOCHHAR
3	De Haan	de haan	DE HAAN
4	Hunold	hunold	HUNOLD
5	Ernst	ernst	ERNST
6	Austin	austin	AUSTIN
7	Pataballa	pataballa	PATABALLA
8	Lorentz	lorentz	LORENTZ

```
SELECT
    last_name,
    lower(last_name) lower,
    upper(last_name) upper

FROM
    employees
```

SUBSTRING

SUBSTRING (expression, start, length)

- **Expression** – ciąg znaków
- **Start** - Jest liczbą całkowitą lub wyrażeniem bigint określającym miejsce rozpoczęcia zwracanych znaków.
 - Pierwszy znak w wyrażeniu to 1.
 - Jeśli wartość start jest mniejsza niż 1, zwrócone wyrażenie rozpocznie się od pierwszego znaku określonego w wyrażeniu.
 - Jeśli wartość start jest większa niż liczba znaków w wyrażeniu wartości, zwracane jest wyrażenie o zerowej długości
- **Length** - Jest dodatnią liczbą całkowitą lub wyrażeniem bigint, które określa, ile znaków wyrażenia zostanie zwróconych.
 - Jeśli wartość length jest ujemna, generowany jest błąd

```
SELECT
    last_name,
    substring(last_name,1,2) substr
FROM
    employees
```

	last_name	substr
1	King	Ki
2	Kochhar	Ko
3	De Haan	De
4	Hunold	Hu
5	Ernst	Er
6	Austin	Au
7	Pataballa	Pa
8	Lorentz	Lo

REPLACE

Zastępuje wszystkie wystąpienia określonej wartości ciągu znaków inną wartością ciągu znaków

```
SELECT
    last_name,
    replace(last_name, 'A', 'XYZ') replace,
    replace(last_name, 'e', 'WWW') replace2
FROM
    employees
```

	last_name	replace	replace2
1	King	King	King
2	Kochhar	KochhXYZr	Kochhar
3	De Haan	De HXYZXYZn	DWWW Haan
4	Hunold	Hunold	Hunold
5	Ernst	Ernst	WWWmst
6	Austin	XYZustin	Austin
7	Pataballa	PXYZiXYZbXYZlXYZ	Pataballa
8	Lorentz	Lorentz	LorWWWntz

TRANSLATE

Zwraca ciąg znaków podany jako pierwszy argument, po przetłumaczeniu niektórych znaków określonych w drugim argumentcie na docelowy zestaw znaków określony w trzecim argumentcie.

TRANSLATE zwróci błąd, jeśli znaki i wyrażenia translacji mają różne długości. TRANSLATE zwróci NULL, jeśli którykolwiek z argumentów ma wartość NULL

```
SELECT
    last_name,
    translate(last_name, 'a', 'X') translate1,
    translate(last_name, 'ao', '12') translate2
FROM
    employees
```

	last_name	translate1	translate2
1	King	King	King
2	Kochhar	KochhXr	K2chh1r
3	De Haan	De HXXn	De H11n
4	Hunold	Hunold	Hun2ld
5	Ernst	Ernst	Ernst
6	Austin	Xustin	1ustin
7	Pataballa	PXtXbXlX	P1t1b1l1
8	Lorentz	Lorentz	L2rentz

REPLICATE

Powtarza wartość ciągu znaków określoną liczbę razy.

```
SELECT  
last_name ,  
REPLICATE('0', 4) + last_name AS 'Last Code'  
FROM employees
```

	last_name	Last Code
1	King	0000King
2	Yang	0000Yang
3	Garcia	0000Garcia
4	James	0000James
5	Miller	0000Miller
6	Williams	0000Williams



REVERSE

Zwraca odwrotną kolejność wartości ciągu znaków

```
SELECT
    last_name,
    reverse(last_name) reverse
FROM
    employees
```

	last_name	reverse
1	King	gniK
2	Kochhar	rahhcok
3	De Haan	naaH eD
4	Hunold	dlonuH
5	Emst	tsnrE
6	Austin	nitsuA
7	Pataballa	allabataP
8	Lorentz	ztneroL

RIGHT, LEFT

Zwraca prawą(RIGHT) lub lewą(LEFT) część ciągu znaków o określonej liczbie znaków.

SELECT

```
last_name,  
RIGHT(last_name,2) r,  
LEFT(last_name,2) l
```

FROM

employees

	last_name	r	l
1	King	ng	Ki
2	Kochhar	ar	Ko
3	De Haan	an	De
4	Hunold	ld	Hu
5	Ernst	st	Er
6	Austin	in	Au
7	Pataballa	la	Pa
8	Lorentz	tz	Lo

CHARINDEX

Wyszukuje jedno wyrażenie znakowe wewnątrz drugiego wyrażenia znakowego, zwracając pozycję początkową pierwszego wyrażenia, jeśli zostanie znalezione

```
select
    last_name,
    charindex ('a', last_name) pozycja_a,
    charindex ('o', last_name) pozycja_o,
    charindex ('u', last_name) pozycja_u
from employees
```

	last_name	pozycja_a	pozycja_u	pozycja_o
1	King	0	0	0
2	Kochhar	6	2	0
3	De Haan	5	0	0
4	Hunold	0	4	2
5	Ernst	0	0	0
6	Austin	1	0	2
7	Pataballa	2	0	0

STUFF

Funkcja STUFF wstawia ciąg znaków do innego ciągu znaków. Usuwa ona określoną długość znaków w pierwszym ciągu na pozycji początkowej, a następnie wstawia drugi ciąg do pierwszego ciągu na pozycji początkowej

```
SELECT  
    STUFF('tekst', 2, 3, ' --NowyTEKST-- '),  
    STUFF('tekst', 3, 1, ' --NowyTEKST-- ')
```

	(No column name)	(No column name)
1	t --NowyTEKST--t	te --NowyTEKST-- st

TRIM

Usuwa znak spacji char(32) lub inne określone znaki z początku i końca ciągu znaków.

```
TRIM ( [ LEADING | TRAILING | BOTH ] [characters FROM ] string )
```

Opcjonalny pierwszy argument określa, która strona łańcucha ma zostać przycięta:

- LEADING usuwa znaki określone od początku ciągu
- TRAILING usuwa znaki określone od końca ciągu
- BOTH (domyślne zachowanie pozycyjne) usuwa znaki określone na początku i na końcu ciągu znaków.
- Domyślnie funkcja TRIM usuwa znak spacji zarówno na początku, jak i na końcu ciągu znaków. To zachowanie jest równoważne funkcji LTRIM(RTRIM(@string)).

```
SELECT
```

```
    TRIM('      test      ') result1,  
    TRIM(' .,! ' FROM ' ! , #      test .') result2,  
    TRIM(LEADING ' .#' FROM '      .#      test !') result3,  
    TRIM(TRAILING ' ! ' FROM '      .#      test ! ') result4
```

	result1	result2	result3	result4
1	test	# test	test !	.# test



Zwraca liczbę znaków(LEN) lub bajtów(DATALENGTH) podanego wyrażenia łańcuchowego, z wyłączeniem końcowych spacji.

```
SELECT
    last_name,
    len(last_name) length ,
    datalength(last_name) dlength
FROM
    employees
```

	last_name	length	dlength
1	King	4	8
2	Kochhar	7	14
3	De Haan	7	14
4	Hunold	6	12
5	Ernst	5	10

LICZBOWE

	round	floor	ceil	modulo	potega	pierwiastek
1	3.1200	3	4	3	64	9

SELECT

```
round(3.1234,2) round,  
floor(3.1234) floor,  
ceiling(3.1234) ceil,  
10%7 modulo,  
power(8,2) potega,  
sqrt(81) pierwiastek
```



YEAR,
MONTH, DAY

YEAR - zwraca liczbę całkowitą reprezentującą rok podanej daty.
Analogicznie MONTH, DAY

```
SELECT  
    hire_date,  
    year(hire_date) rok,  
    month(hire_date) miesiac,  
    day(hire_date) dzien  
from employees
```

	hire_date	rok	miesiac	dzien
1	1987-06-17	1987	6	17
2	1989-09-21	1989	9	21
3	1993-01-13	1993	1	13
4	1990-01-03	1990	1	3
5	1991-05-21	1991	5	21
6	1997-06-25	1997	6	25
7	1998-02-05	1998	2	5
8	1999-02-07	1999	2	7

DATEDIFF

Zwraca liczbę określonych wartości między określoną datą początkową i końcową.

```
SELECT
    hire_date,
    getdate(),
    DATEDIFF(year, hire_date, getdate()) result_year,
    DATEDIFF(month, hire_date, getdate()) result_month,
    DATEDIFF(day, hire_date, getdate()) result_day
from employees
```

	hire_date	(No column name)	result_year	result_month	result_day
1	1987-06-17	2021-05-19 14:09:00.280	34	407	12390
2	1989-09-21	2021-05-19 14:09:00.280	32	380	11563
3	1993-01-13	2021-05-19 14:09:00.280	28	340	10353
4	1990-01-03	2021-05-19 14:09:00.280	31	376	11459
5	1991-05-21	2021-05-19 14:09:00.280	30	360	10956
6	1997-06-25	2021-05-19 14:09:00.280	24	287	8729
7	1998-02-05	2021-05-19 14:09:00.280	23	279	8504
8	1999-02-07	2021-05-19 14:09:00.280	22	267	8137

DATE_ADD

Dodaje liczbę do części daty wejściowej i zwraca zmodyfikowaną wartość daty/czasu

Wszystkie możliwości: <https://learn.microsoft.com/en-us/sql/t-sql/functions/dateadd-transact-sql?view=sql-server-ver16>

```
SELECT
```

```
    hire_date,  
    DATEADD(year, 2, hire_date) result_year,  
    DATEADD(month, 2, hire_date) result_month,  
    DATEADD(day, 2, hire_date) result_day  
from employees
```

	hire_date	result_year	result_month	result_day
1	1987-06-17	1989-06-17	1987-08-17	1987-06-19
2	1989-09-21	1991-09-21	1989-11-21	1989-09-23
3	1993-01-13	1995-01-13	1993-03-13	1993-01-15
4	1990-01-03	1992-01-03	1990-03-03	1990-01-05
5	1991-05-21	1993-05-21	1991-07-21	1991-05-23
6	1997-06-25	1999-06-25	1997-08-25	1997-06-27
7	1998-02-05	2000-02-05	1998-04-05	1998-02-07
8	1999-02-07	2001-02-07	1999-04-07	1999-02-09

GETDATE

GETDATE - Zwraca bieżący znacznik czasu systemu bazy danych jako wartość datetime bez przesunięcia strefy czasowej bazy danych.

Wartość ta pochodzi z systemu operacyjnego komputera, na którym uruchomiona jest instancja SQL Server.

GETUTCDATE - Zwraca bieżący znacznik czasu systemu bazy danych jako wartość datetime. Przesunięcie strefy czasowej bazy danych nie jest uwzględniane. Wartość ta reprezentuje bieżący czas UTC (Coordinated Universal Time).

SYSDATETIME - Zwraca wartość zawierającą datę i godzinę komputera, na którym uruchomiona jest instancja SQL Server.

CURRENT_TIMESTAMP - Ta funkcja zwraca bieżący znacznik czasu systemu bazy danych jako wartość datetime, bez przesunięcia strefy czasowej bazy danych.

SELECT

```
GETDATE() gd,  
GETUTCDATE() gud,  
SYSDATETIME() sdt,  
CURRENT_TIMESTAMP cts
```

	gd	gud	sdt	cts
1	2024-06-14 16:48:53.767	2024-06-14 14:48:53.767	2024-06-14 16:48:53.7670495	2024-06-14 16:48:53.767

FORMAT

	Date	Custom Number	US English	British English
1	04/12/2023	123-45-6789	Monday, December 4, 2023	04 December 2023

SELECT

```
FORMAT( getdate(), 'dd/MM/yyyy' ) AS Date  
, FORMAT(123456789, '###-##-####') AS CustomNumber  
, FORMAT( getdate(), 'D', 'en-US' ) USEnglish  
, FORMAT( getdate(), 'D', 'en-gb' ) BritishEnglish
```

<https://www.sqlshack.com/a-comprehensive-guide-to-the-sql-format-function/>

CONVERT DATE

	(No column name)	(No column name)	(No column name)
1	05/19/2021	2021.05.19	19.05.2021

```
SELECT
    convert(varchar, getdate(), 101),
    convert(varchar, getdate(), 102),
    convert(varchar, getdate(), 104)
```

CONVERT VARCHAR

	(No column name)	(No column name)
1	2	11

```
SELECT
    1 + 1,
    convert(varchar,1) + convert(varchar,1),
    CAST(1 as varchar) + CAST(1 as varchar)
```

SQL_VARIANT

Sql_variant może być używany w kolumnach, parametrach, zmiennych i wartościach zwracanych funkcji zdefiniowanych przez użytkownika.

Kolumna typu sql_variant **może zawierać wiersze różnych typów danych**. Na przykład kolumna zdefiniowana jako sql_variant może przechowywać wartości int, binary i char. Może mieć maksymalną długość 8016 bajtów.

```
drop table t1
```

```
create table t1 (a SQL_VARIANT)
```

```
insert into t1 values (1)
```

```
insert into t1 values ('a')
```

```
insert into t1 values (CURRENT_TIMESTAMP)
```

```
select
```

```
t1.*,
```

```
SQL_VARIANT_PROPERTY(t1.a, 'BaseType')
```

```
from t1
```

	a	typ_danych
1	1	int
2	a	varchar
3	2025-04-06 14:58:01.900	datetime

SQL_VARIANT _PROPERTY

Zwraca podstawowy typ danych i inne informacje o wartości sql_variant.

```
drop table t1
```

```
CREATE TABLE t1(a sql_variant, b int)
```

```
INSERT INTO t1 values ( cast(46279.1 as decimal(8,2)), 777)
```

```
SELECT SQL_VARIANT_PROPERTY(a, 'BaseType') AS 'Base Type',  
       SQL_VARIANT_PROPERTY(a, 'Precision') AS 'Precision',  
       SQL_VARIANT_PROPERTY(a, 'Scale') AS 'Scale'
```

```
FROM t1
```

```
WHERE b = 777
```

	Base Type	Precision	Scale
1	decimal	8	2